ANNASAHEB DANGE COLLEGE OF ENGINEERING AND TECHNOLOGY, ASHTA

(An Empowered Autonomous Institute)



Department of Artificial Intelligence & Data Science

REPORT
ON

TICKETFLOW – NLP-BASED SUPPORT TICKET CLASSIFICATION SYSTEM

A Python-Based NLP Micro-Project using TF-IDF and Logistic Regression

Subject: NLP

Academic Year: 2026 – 2027

Submitted By :-

ROLL No	Name	URN
4048	Abhishek Rajendrakumar Ainapure	23091043
4055	Samarth Satish Pawar	23091055

ABSTRACT

Customer complaints arrive in large numbers through web forms, e-mails and chat, and each one must be understood, categorized, prioritized and sent to the correct team. Doing this manually is slow and inconsistent, and urgent problems can stay hidden in a long queue. Automated text classification can reduce this effort and improve the speed of customer support.

TicketFlow is a Python-based Natural Language Processing (NLP) micro-project that automatically classifies customer complaints into four categories: Technical, Billing, Account and Service. Complaint text is cleaned and converted into numerical features using TF-IDF, and a Logistic Regression model from Scikit-learn predicts the category. Keyword-based rules detect the priority of the complaint as High, Medium or Low, and the ticket is assigned automatically to the responsible department.

The application is delivered through a professional black-and-yellow Streamlit dashboard with user login and signup. Every processed ticket is stored in an SQLite database, and the ticket history can be viewed and exported as a CSV file. Pandas is used for data handling. This project demonstrates how NLP and machine learning can be combined with a simple web interface to automate ticket classification, priority assignment and department routing.

Keywords: NLP, Text Classification, TF-IDF, Logistic Regression, Scikit-learn, Streamlit, SQLite, Pandas, Support Ticket, Priority Detection.

TABLE OF CONTENTS

Sr. No.	Contents	Page No.
1	**Chapter 1: Introduction**	**4**
1.1	Introduction	4
1.2	Problem Statement	4
1.3	Objectives	4
1.4	Scope	5
2	**Chapter 2: Literature Review and Existing System**	**5**
2.1	Literature Review	5
2.2	Existing System	6
2.3	Proposed System	6
2.4	Comparison of Approaches	6
3	**Chapter 3: Proposed System and Methodology**	**6**
3.1	System Overview	6
3.2	System Workflow	6
3.3	Text Preprocessing	7
3.4	Tokenization	7
3.5	TF-IDF Vectorization	7
3.6	Logistic Regression Classification	8
3.7	Machine Learning Pipeline	8
3.8	Priority Detection	8
3.9	Department Assignment	9
3.10	User Login and Signup	9
4	**Chapter 4: System Design and Architecture**	**9**
4.1	System Architecture	9
4.2	Use Case Diagram	10
4.3	Data Flow Diagram	10
4.4	Database Design	10
4.5	Module Design	11
4.6	User Interface Design	11
5	**Chapter 5: System Implementation**	**11**
5.1	Technologies Used	11
5.2	Dataset and Model Training	11
5.3	NLP Module	11
5.4	Priority Detection Module	12
5.5	Department Assignment Module	12
5.6	Authentication Module	12
5.7	Database Implementation	12
5.8	Streamlit Dashboard Implementation	12
5.9	CSV Export	12
5.10	Installation and Execution	12
6	**Chapter 6: Testing and Results**	**13**
6.1	Testing	13
6.2	Test Cases	13
6.3	Performance Evaluation	13
6.4	Results and Discussion	14
7	**Chapter 7: Conclusion and Future Scope**	**14**
7.1	Conclusion	14
7.2	Limitations	14
7.3	Future Scope	15
8	**References**	**15**

CHAPTER 1: INTRODUCTION

1.1 Introduction

Every organization that serves customers receives complaints. These complaints arrive through web forms, e-mails, chat windows and phone calls, and each one has to be read, understood, categorized, prioritized and sent to the right team before anyone can start solving it. When the number of complaints is small, this work is done by hand without much difficulty. As the number of customers grows, however, manual handling becomes slow, inconsistent and expensive, and urgent problems can remain hidden in a long queue of routine requests.

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to read and analyze human language. Combined with supervised machine learning, NLP can learn from complaints that were already categorized by people and then apply the same judgement to new complaints within a fraction of a second. This idea is known as automatic text classification and it is the foundation of modern support-ticket automation.

TicketFlow is a Python-based NLP micro-project that automatically classifies customer complaints into four categories: **Technical, Billing, Account and Service**. It converts the complaint text into numerical features using **TF-IDF** (Term Frequency – Inverse Document Frequency) and predicts the category using **Logistic Regression** from the Scikit-learn library. After classification, the system detects the priority of the complaint using keyword-based rules and assigns the ticket to the responsible department automatically.

1.2 Problem Statement

Customer-support teams receive complaints in free text. Reading each message, deciding which kind of problem it describes, judging how urgent it is and forwarding it to the correct department takes time and depends on the experience of the person doing it. Different agents may classify the same complaint differently, urgent complaints may wait in the same queue as simple queries, and wrongly routed tickets are passed between departments before they reach someone who can solve them.

The main difficulties of manual ticket handling are:

- **Delay:** every ticket must be read and routed by a person before work can begin.
- **Inconsistency:** the category and priority depend on the individual judgement of the agent.
- **Misrouting:** tickets sent to the wrong department increase the resolution time.
- **Poor visibility:** urgent complaints are not highlighted automatically.
- **No history:** without a central record it is difficult to review or export past tickets.

Therefore, an automated system is required that can read a complaint, classify it into a suitable category, detect its priority, assign it to a department and keep a searchable history of all tickets.

1.3 Objectives

1. To develop a user-friendly support-ticket classification application using Python and Streamlit.
2. To apply NLP preprocessing techniques to customer complaint text.
3. To convert text into numerical features using the TF-IDF technique.
4. To classify complaints into Technical, Billing, Account and Service categories using Logistic Regression.
5. To detect the priority (High, Medium or Low) of a complaint using keyword-based rules.
6. To assign each ticket automatically to the responsible department.
7. To provide secure user login and signup.
8. To store ticket history in an SQLite database and export it as a CSV file.

1.4 Scope

The system focuses on English-language customer complaints and classifies them into four fixed categories. It covers text preprocessing, feature extraction, supervised classification, rule-based priority detection, department routing, user authentication, ticket storage, history viewing and CSV export through a web dashboard.

It is useful for small and medium businesses, student projects, help-desk teams and any organization that wants a lightweight way to organize customer complaints. The system does not guarantee a correct prediction for every possible complaint: its accuracy depends on the quality and size of the training data, and the priority rules cover only the implemented keywords.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEM

2.1 Literature Review

Text classification is one of the oldest and most widely used applications of Natural Language Processing. The task is to assign a document to one or more predefined classes according to its content. Typical examples are spam detection, sentiment analysis, news categorization and, in the context of this project, support-ticket routing [1][12].

In information retrieval, the importance of a word in a document is measured by weighting schemes. Spärck Jones introduced the idea of inverse document frequency, which gives higher weight to rare terms, and Salton and Buckley later studied term-weighting approaches that combined term frequency with inverse document frequency. The resulting TF-IDF weighting is still a standard way of representing text for classical machine-learning algorithms [2][3].

Several algorithms have been used with TF-IDF features. Naive Bayes is fast and works well on small datasets. Support Vector Machines were shown by Joachims to perform strongly on high-dimensional text data [4]. Logistic Regression is a linear model that predicts class probabilities and is easy to train, interpret and deploy, which makes it a popular baseline for text classification [5][6]. More recently,

transformer-based language models such as BERT have achieved very high accuracy on many text tasks [7], but they need larger datasets, more computing power and more complex deployment.

2.2 Existing System

In many small organizations, complaints are still handled manually. An agent reads each message, decides the category, estimates the urgency and forwards it to a department by e-mail or by editing a spreadsheet. This approach needs no technology, but it is slow and its quality changes from person to person.

2.3 Proposed System

The proposed system, TicketFlow, uses a lightweight machine-learning pipeline. Complaint text is cleaned, converted into TF-IDF vectors and classified by a Logistic Regression model into Technical, Billing, Account or Service. A separate keyword-based module assigns High, Medium or Low priority, and a mapping table assigns the ticket to a department.

2.4 Comparison of Approaches

Table 2.1: Comparison of Approaches

Approach	Advantage	Limitation
Manual triage	Human understanding of context	Slow, inconsistent and costly
Keyword rules only	Simple and explainable	Fails on new wording; hard to maintain
Naive Bayes	Very fast; works with small data	Assumes words are independent
Support Vector Machine	Strong accuracy on text	Gives no direct probabilities; slower to tune
Deep learning / transformers	Understands context deeply	Needs large data and more resources
TicketFlow (TF-IDF + Logistic Regression + rules)	Fast, explainable, gives probabilities, runs on a normal PC	Limited to trained categories; ignores deep context

CHAPTER 3: PROPOSED SYSTEM AND METHODOLOGY

3.1 System Overview

The application accepts a customer complaint from an authenticated user, processes it through an NLP and machine-learning pipeline, and returns the category, priority and department through the dashboard. The complete workflow is shown in Figure 3.1.

3.2 System Workflow

Figure 3.1 shows the complete workflow of TicketFlow from user login to history export.

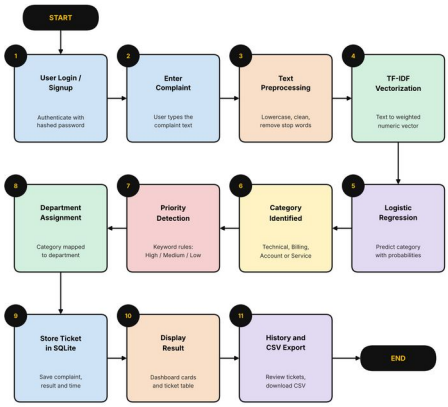


Figure 3.1: System Workflow of TicketFlow

3.3 Text Preprocessing

Raw complaint text contains capital letters, punctuation, numbers, links and very common words that carry little meaning. Text preprocessing prepares the input so that the model can focus on the words that identify the problem. The preprocessing steps used in TicketFlow are:

1. **Lowercasing:** “Refund” and “refund” are treated as the same word.
2. **Noise removal:** web links, extra spaces, digits and punctuation marks are removed.
3. **Stop-word removal:** very common words such as “the”, “is” and “of” are removed. Negation words such as “not” and “no” are retained because they change the meaning of phrases like “not working”.
4. **Whitespace normalization:** the remaining words are joined with single spaces.

3.4 Tokenization

Tokenization divides the cleaned text into smaller units called tokens. In TicketFlow the tokens are words, and the vectorizer also forms two-word combinations (bigrams) so that useful phrases such as “not working” or “charged twice” are captured as single features.

3.5 TF-IDF Vectorization

A machine-learning model cannot work directly with text, so every complaint must be converted into a vector of numbers. TF-IDF (Term Frequency – Inverse Document Frequency) is a weighting technique that gives a high value to words that are frequent in a complaint but rare in the full collection of complaints.

Term Frequency (TF) measures how often a term t appears in a document d. **Inverse Document Frequency (IDF)** measures how rare the term is across all N documents. In Scikit-learn the smoothed form is used:

$$idf(t) = ln[(1 + N) / (1 + df(t))] + 1 \qquad tf\text{-}idf(t, d) = tf(t, d) \times idf(t)$$

Here df(t) is the number of complaints that contain the term t. The resulting vectors are normalized so that long and short complaints can be compared fairly. Words such as “refund”, “password” or “crash”

receive high weights because they strongly indicate one category, whereas words that appear in all categories receive low weights.

3.6 Logistic Regression Classification

Logistic Regression is a supervised learning algorithm used for classification. For more than two classes it uses the **multinomial (softmax)** form. For a TF-IDF vector x, the model computes one linear score for every class k and converts the scores into probabilities:

$$z_k = w_k \cdot x + b_k \qquad P(y = k \mid x) = \exp(z_k) / \Sigma_j \exp(z_j)$$

The four probabilities add up to one, and the predicted category is the class with the highest probability. During training, the weights w and biases b are learned by minimizing the log-loss on the labelled complaints; L2 regularization is applied by default in Scikit-learn to prevent over-fitting.

Logistic Regression was chosen for TicketFlow because it is fast to train, works well with sparse TF-IDF features, gives class probabilities, and its learned weights can be inspected to understand which words influence each category.

3.7 Machine Learning Pipeline

The machine-learning part of TicketFlow has two phases. In the **training phase**, a labelled dataset of complaints is cleaned, split into training and testing sets, vectorized with TF-IDF and used to fit the Logistic Regression model. The model and the vectorizer are evaluated and then saved to disk. In the **prediction phase**, the saved files are loaded and every new complaint passes through exactly the same cleaning and vectorization steps before the model predicts its category.

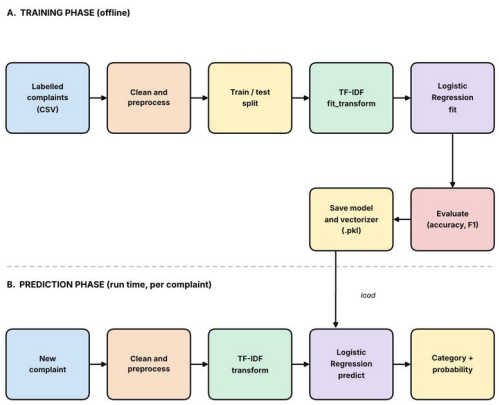


Figure 3.2: Training and Prediction Pipeline

Using the same vectorizer in both phases is essential: it guarantees that a word receives the same feature position and weight during training and prediction.

3.8 Priority Detection

Priority is not predicted by the machine-learning model. It is detected by transparent keyword-based rules so that the support team can understand and modify them easily. The complaint text is converted to lowercase and checked against two keyword lists. If a high-priority keyword is found, the ticket is

marked **High**; otherwise, if a medium-priority keyword is found, it is marked **Medium**; if neither is found, the ticket is **Low**.

Table 3.2: Priority Levels and Sample Keywords

Priority	Meaning	Sample Keywords
High	Needs immediate attention	urgent, immediately, asap, critical, emergency, fraud, hacked, unauthorized, crash, not working, charged twice, data loss
Medium	Should be solved soon	slow, delay, error, issue, problem, refund, failed, pending, wrong, cannot, unable
Low	Routine or informational	No high or medium keyword found

3.9 Department Assignment

After the category is predicted, the ticket is assigned to the department responsible for that kind of problem using a simple mapping. Routing does not need a second model because each category corresponds to one department.

Table 3.3: Category-to-Department Mapping

Predicted Category	Assigned Department	Responsibility
Technical	Technical Support Team	Application, website and device faults
Billing	Finance and Billing Department	Payments, invoices and refunds
Account	Account Management Team	Login, password and profile problems
Service	Customer Service Department	Delivery, response and staff-related issues

3.10 User Login and Signup

Only registered users can submit tickets. During signup, the application checks that the username and e-mail are not already registered, converts the password into a hash and stores the new user in the database. During login, the entered password is hashed in the same way and compared with the stored value. The password itself is never stored in plain text. After a successful login, the user session is stored in Streamlit's session state and the dashboard is opened.

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture

TicketFlow follows a three-layer architecture. The **presentation layer** is the Streamlit dashboard through which the user logs in, submits complaints and views results. The **application layer** contains the Python modules for authentication, preprocessing, TF-IDF vectorization, Logistic Regression classification and

priority detection. The **data layer** consists of the SQLite database, the saved model files and the CSV files used for training and export.

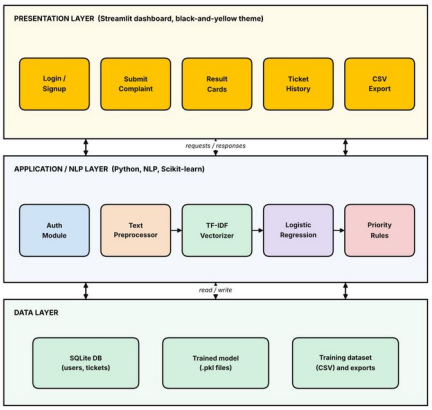


Figure 4.1: System Architecture of TicketFlow

4.2 Use Case Diagram

The main actor is the user, who may be a customer-support agent or a customer representative entering complaints. The system itself (NLP model and database) acts as a supporting actor. Submitting a complaint includes the classification, priority detection and department assignment steps.

4.3 Data Flow Diagram

Level 0. The Level 0 diagram shows TicketFlow as a single process. The user sends login details and complaint text and receives the category, priority and department. The system saves tickets and user records in the SQLite database and reads history from it.

Level 1. The Level 1 diagram divides the system into six processes: authentication (1.0), text preprocessing (2.0), vectorization and classification (3.0), priority detection (4.0), department assignment (5.0) and ticket storage and retrieval (6.0). Two data stores are used: D1 for users and D2 for tickets.

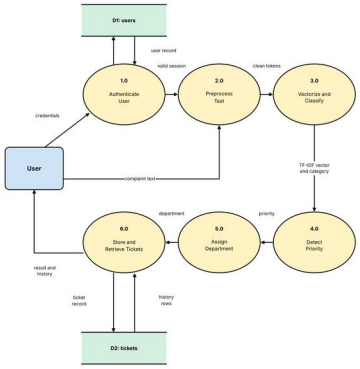


Figure 4.2: Data Flow Diagram – Level 1

4.4 Database Design

SQLite is used for persistent storage. The database contains two related tables: **users**, which stores registered accounts, and **tickets**, which stores every processed complaint together with its category, priority and department. One user can submit many tickets, so the relationship is one-to-many.

4.5 Module Design

The source code is divided into small modules, each with a single responsibility. The main file, app.py, builds the Streamlit interface and calls the other modules. This separation makes the project easy to understand, test and extend.

Table 4.1: Responsibilities of the Modules

Module	Responsibility	Main Functions
app.py	Streamlit pages, theme and navigation	show_auth(), show_dashboard()
auth.py	Password hashing and verification	hash_password(), verify_password()
database.py	SQLite connection and queries	init_db(), create_user(), add_ticket(), get_tickets()
preprocess.py	Text cleaning	clean_text()
model.py	Training, loading and prediction	train_model(), predict_category()
priority.py	Priority rules and department mapping	detect_priority(), assign_department()

4.6 User Interface Design

The dashboard uses a professional **black-and-yellow** theme. A dark background reduces eye strain during long support sessions, and the yellow accent draws attention to headings, buttons and result cards. The sidebar provides navigation, while the main area contains the complaint form and the result cards.

CHAPTER 5: SYSTEM IMPLEMENTATION

5.1 Technologies Used

The project is built with Python. Scikit-learn provides the TF-IDF vectorizer, the Logistic Regression classifier and the evaluation metrics; Pandas handles data and CSV export; SQLite stores users and tickets; Joblib saves the trained model; and Streamlit provides the black-and-yellow web dashboard.

5.2 Dataset and Model Training

The classifier is trained on a CSV file in which every row contains a complaint and its correct category. The file has two main columns, complaint and category, and should contain a balanced number of examples for Technical, Billing, Account and Service so that the model does not favour one class. The dataset is split into 80% training and 20% testing data using a stratified split so that all categories appear in both parts.

5.3 NLP Module

The NLP module cleans the complaint text before vectorization. It lowercases the text, removes links, digits and punctuation, and removes stop words while keeping negation words. The prediction function

then loads the saved vectorizer and model, transforms the cleaned text and returns the predicted category with its probability.

5.4 Priority Detection Module

The priority module keeps two keyword lists and checks them with regular expressions that match the beginning of a word. This avoids false matches inside unrelated words and still recognizes forms such as “crash”, “crashes” and “crashed”.

5.5 Department Assignment Module

Department assignment is a dictionary lookup from the predicted category to the department name. An unknown category falls back to the general customer-service department.

5.6 Authentication Module

Passwords are converted into salted hashes before they are stored. During login, the entered password is hashed with the stored salt and compared with the saved hash.

5.7 Database Implementation

The database module uses Python’s built-in sqlite3 library. It creates the tables when the application starts, inserts new users and tickets with parameterized queries (which protect against SQL injection) and reads the ticket history of the logged-in user into a Pandas DataFrame [11].

5.8 Streamlit Dashboard Implementation

The black-and-yellow look is created in two ways. First, the Streamlit theme is set in the file .streamlit/config.toml. Second, a small block of custom CSS styles the buttons, result cards and tables.

The main page checks whether a user is logged in. If not, the login and signup forms are shown; otherwise the dashboard opens. When the Classify button is pressed, the application runs the complete pipeline, shows the result cards and saves the ticket.

5.9 CSV Export

The history page shows the saved tickets in a table and offers a download button. The DataFrame returned by the database module is converted to CSV text and passed to Streamlit’s download button.

5.10 Installation and Execution

1. Install Python 3.9 or later.
2. Install the dependencies: pip install streamlit scikit-learn pandas joblib
3. Train the model once: python -c "from model import train_model; train_model()"
4. Start the application: streamlit run app.py
5. Open the local URL shown in the terminal, create an account and begin submitting complaints.

CHAPTER 6: TESTING AND RESULTS

6.1 Testing

Testing verifies that the application behaves as expected. The following types of testing are applicable to TicketFlow:

- **Unit testing:** individual functions such as clean_text(), detect_priority() and assign_department() are tested with known inputs.
- **Model testing:** the classifier is evaluated on the held-out test set.
- **Functional testing:** signup, login, classification, history and export are tested through the dashboard.
- **Database testing:** records are inserted and retrieved correctly, and duplicate usernames are rejected.
- **Interface testing:** the layout, theme and result cards are checked on different screen sizes.

6.2 Test Cases

Table 6.1: Sample Test Cases

ID	Input / Action	Expected Behaviour
TC01	The mobile app crashes every time I open it.	Technical; High; Technical Support Team
TC02	I was charged twice for my subscription. Please refund immediately.	Billing; High; Finance and Billing
TC03	I forgot my password and cannot log in to my account.	Account; Medium; Account Management
TC07	Someone hacked my account and changed my e-mail address.	Account; High; Account Management
TC09	Empty complaint box and Classify clicked	Warning shown; no ticket saved
TC10	Signup with an existing username	Error shown; no duplicate user created
TC11	Login with a wrong password	Login rejected with an error message
TC12	Open History and click Export CSV	CSV file with all saved tickets is downloaded

Record the actual Pass/Fail result after executing each test in your application.

6.3 Performance Evaluation

The classifier should be evaluated on the held-out 20% test set using standard metrics. For each category, the numbers of true positives (TP), false positives (FP) and false negatives (FN) are obtained from the confusion matrix.

Table 6.3: Evaluation Metrics

Metric	Formula	Meaning
Accuracy	(Correct predictions) / (All predictions)	Overall share of correctly classified complaints
Precision	TP / (TP + FP)	How many predicted tickets of a class are correct
Recall	TP / (TP + FN)	How many actual tickets of a class are found
F1-score	2 × P × R / (P + R)	Balance between precision and recall

In Scikit-learn, these values are produced together by classification_report() and the confusion matrix by confusion_matrix(). Stratified k-fold cross-validation can be used to confirm that the result does not depend on one particular split.

6.4 Results and Discussion

TicketFlow integrates NLP preprocessing, TF-IDF feature extraction, a Logistic Regression classifier, keyword-based priority rules, department routing, authentication and persistent storage into a single practical application. The user receives the category, priority and department immediately after submitting a complaint, and every ticket is saved for later review.

The combination of a machine-learning model for category and transparent rules for priority gives a good balance: the model handles the variety of customer wording, while the rules make urgency easy to understand and adjust. The history and CSV export features extend the project beyond a simple classifier and make it useful as a small ticket-management tool. The final evaluation should be based on actual test results and measured experiments on the project dataset.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

TicketFlow demonstrates how Natural Language Processing and machine learning can simplify customer-complaint management. The system cleans complaint text, converts it into TF-IDF vectors and classifies it into Technical, Billing, Account or Service using Logistic Regression. It then detects priority with keyword-based rules and assigns the ticket to the correct department.

Streamlit provides a professional black-and-yellow dashboard, user login and signup protect access, and SQLite stores the complete ticket history, which can be exported as a CSV file. The project shows that a lightweight, explainable pipeline built with Python, Scikit-learn, Pandas and SQLite can automate classification, priority assignment and routing, and it provides a useful foundation for larger help-desk solutions.

7.2 Limitations

- The system handles only English text and four fixed categories.

- Accuracy depends on the size and quality of the training dataset.
- TF-IDF does not capture deep meaning, sarcasm or complex context.
- Priority rules cover only the implemented keywords and may miss unusual wording.
- A complaint that covers several issues receives only one category.
- SQLite is suitable for small deployments but not for very large multi-user systems.

7.3 Future Scope

- Transformer-based models such as BERT for deeper understanding.
- Machine-learning-based priority prediction instead of keyword rules.
- Multilingual complaint support.
- Role-based access with an administrator and agent dashboard.
- E-mail and chat integration for automatic ticket creation.
- Analytics charts for category and priority trends.
- Sentiment analysis and automatic reply suggestions.
- Cloud deployment with a larger database and stronger password hashing.

REFERENCES

1. S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. O'Reilly Media, 2009.
2. K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," Journal of Documentation, vol. 28, no. 1, pp. 11–21, 1972.
3. G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.
4. T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in Proc. European Conference on Machine Learning (ECML), 1998, pp. 137–142.
5. D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Wiley, 2013.
6. C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008.
7. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.
8. F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Available: https://scikit-learn.org/
9. Streamlit Documentation. Available: https://docs.streamlit.io/
10. SQLite Documentation. Available: https://www.sqlite.org/docs.html

11. W. McKinney, “Data structures for statistical computing in Python,” in Proc. 9th Python in Science Conference, 2010, pp. 56–61.
12. D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. (draft). Available: https://web.stanford.edu/~jurafsky/slp3/
13. Python Software Foundation, “Python Documentation.” Available: https://docs.python.org/3/